VIBECHAIN

The Ultimate Blockchain Challenge | MERAZ 7.0 | BIB x DSC

PROBLEM STATEMENT

Trustworthy Club Treasury

Event	VIBECHAIN – The Ultimate Blockchain Challenge (DSC × BIB) at MERAZ 7.0
Date	1 November 2026
Time	10:00 AM – 1:00 PM (Round 1: Build – 2 hrs | Round 2: Break – 1 hr)
Venue	L-101, LHC, IIT Bhilai
Team Size	1 – 3 members
Prize Pool	Worth ₹5,000

1. Background

Clubs, student organisations and communities pool money for events, equipment, prizes and trips. In practice, this money lives in a bank account or a UPI handle controlled by one or two people. Decisions are made in private chats, receipts get lost, and months later nobody can say with confidence who approved what, where the money went, or whether it was worth it.

Blockchains let rules be enforced by code instead of by trust, and let every action be publicly verifiable.

2. The Challenge

Design and build a decentralized application (dApp) that makes a club's shared funds transparent, accountable and hard to abuse.

Your solution should make it clear to a judge :

- **Who** is allowed to do what.
- **How** a decision about money gets made.
- **Why** no single person, including you as the deployer, can quietly misuse the funds.
- **What** a member can verify publicly afterwards.

3. Core Requirements (Must-Have)

These describe outcomes, not implementations. How you achieve them is up to you.

3.1 Smart Contract

Area	Requirement
Treasury	Holds the club's funds (test ETH or test tokens). Anyone can contribute by sending test funds.

Area	Requirement
Participation	The system must define who counts as a member or participant and how someone becomes one (or stops being one).
Decision-making	Spending of treasury funds must be governed by on-chain rules that you design. Those rules must be enforced by the contract, not by a person.
Accountability	Every outflow of funds must be publicly traceable on-chain to a stated purpose and a decision that authorised it.
No backdoors	No admin, owner or deployer may be able to move funds outside the rules the contract defines.

Safety invariants (mandatory, whatever design you choose):

- The contract can never pay out more than it holds.
- No authorisation can be used to move funds more than it was granted for.
- An action that was rejected, expired or never authorised must not move funds.

3.2 Frontend (Website)

- Wallet connection using wallet (MetaMask).
- A dashboard that gives an outsider a clear picture of the treasury: its balance, who participates, and the history and current state of how money is being decided on and spent.
- Interfaces for every action your design allows a participant to take.
- Clear feedback on transaction status (pending, success, failed) and clear errors when an action is not permitted.

3.3 Deployment

- Deploy the smart contract on an Ethereum testnet (Sepolia recommended).
- Provide a working live demo of the dApp (any deployment service is permitted; Vercel is one option).

4. Technology Requirements

Area	Requirement
Development environment	Antigravity only (Round 1). Gemini is additionally permitted in Round 2.
Smart contracts	Solidity (Hardhat or Foundry suggested).
dApp / Frontend	Team's choice of technology (React / Next.js with ethers.js or wagmi/viem suggested).
Blockchain	Ethereum testnet (Sepolia recommended).
Wallet / Transactions	MetaMask.
AI usage	Only models available inside Antigravity. Using any other AI tool where prohibited may result in a penalty or disqualification.

5. Event Format

Round 1 – Build (First 2 Hours)

Build the complete full-stack dApp, deploy it, document it, and submit your public GitHub repository link through the official submission form before the Round 1 deadline.

Round 2 – Break the Bugs (Final Hour)

Inspect the public repositories of other teams, identify bugs and security vulnerabilities (including flaws in their *design*, not only their code), document them clearly, and submit your findings through the official reporting form. Think like both a developer and a security tester.

6. Deliverables

1. **Public GitHub repository** with the full source code (smart contract and frontend), submitted from your single team GitHub account.
2. **Documentation** (README) in the repository covering: project overview, the problem you chose to solve, architecture, setup steps, deployed contract address and network, live demo link, and **a justification of design choices** you made.
3. **Live demo** link of the deployed dApp connected to the testnet contract.
4. **Vulnerability report** (Round 2) submitted through the official reporting form.

7. Judging Criteria

Criterion	Weight
Core Functionality (including soundness and originality of your design)	30%
UI & Design	20%
Development Documentation	15%
Vulnerability Discovery	15%
Vulnerability Documentation	15%
Deployment	5%
Total	**100%**

The detailed points system and scoring method will be shared later through the official event channels.

8. Important Notes

- Each team must use one GitHub account and keep the submission repository public.
- Ensure every submitted link works and is accessible to reviewers.
- All submissions are made on the spot using links shared through the official event channels.
- Event rules may be updated at any time; changes will be announced in the official event group.
- The decisions of the event committee and judges are final.

All the best, and happy building!